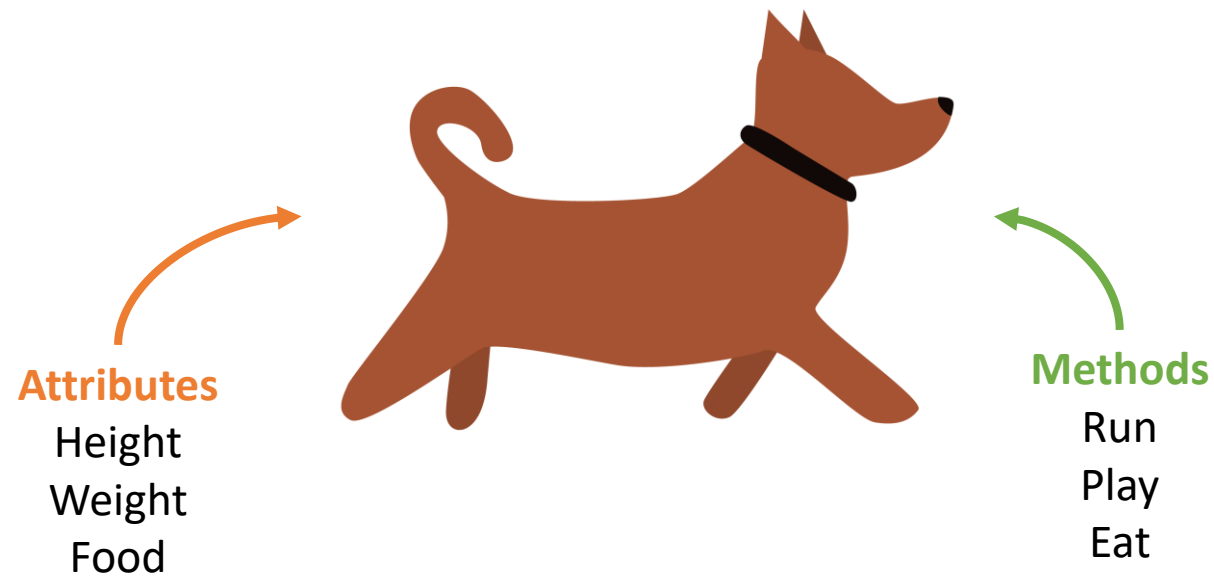


MOBILE DEVELOPMENT

W2 – Dart – OOP

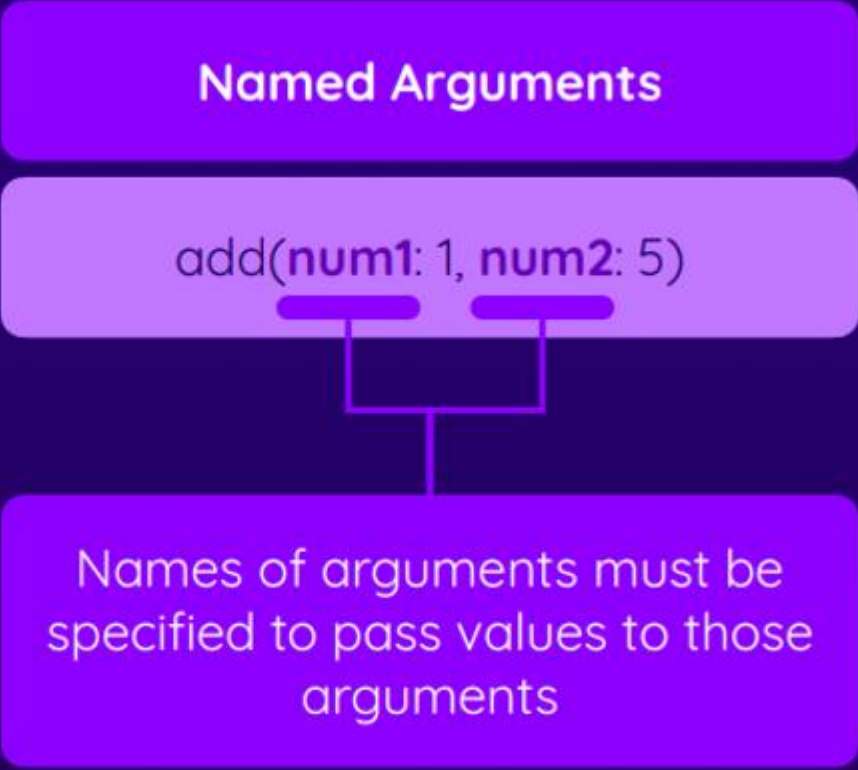


CADT
IDT

Dart arguments can be 'named' or 'positional'

Named Arguments

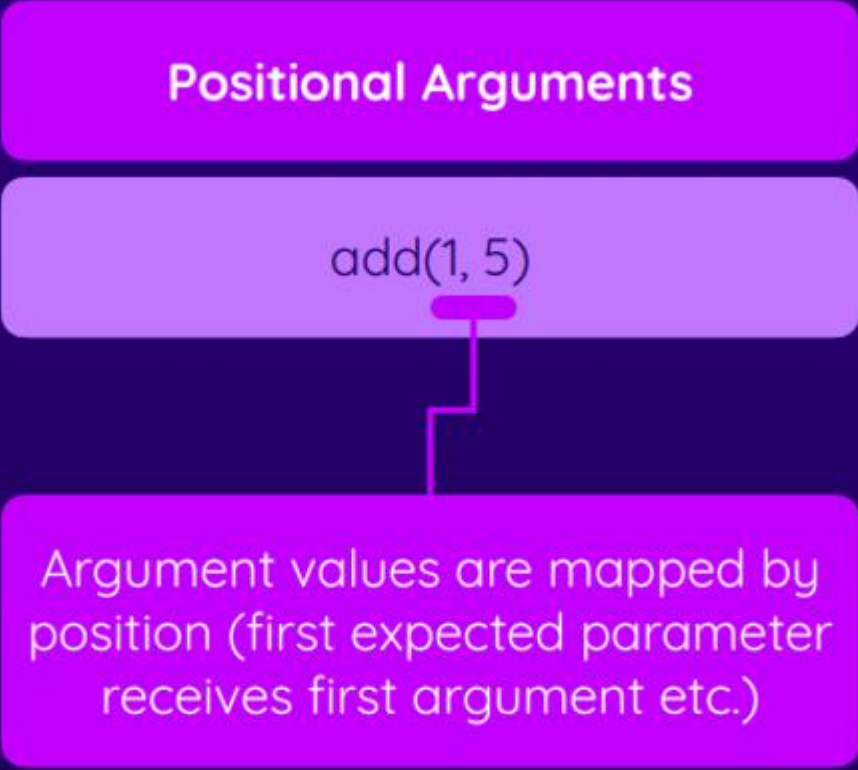
```
add(num1: 1, num2: 5)
```



Names of arguments must be specified to pass values to those arguments

Positional Arguments

```
add(1, 5)
```



Argument values are mapped by position (first expected parameter receives first argument etc.)

Let's implement together the bellow **function**

Description

Name	printStudent
Parameters	Name Age (default is 20) Address (optional) Score (a list of scores)
Return	void
Description	print the student information

Example

```
printStudent(name:"ronan", scores: [20,15,45])
```



```
Student      : Ronan
Age          : 20
Address      : No address
Scores       : 15 20 45
```

ANSWER

```
void printStudent({  
    required String name,  
    int age = 20,  
    String? address,  
    required List<int> scores,  
}) {  
    // Name  
    print("Student\t\t: $name");  
  
    // Age  
    print("Age\t\t: $age");  
  
    // Address  
    String addressValue;  
    if (address != null) {  
        addressValue = address;  
    } else {  
        addressValue = "No address";  
    }  
    print("Address\t\t: $addressValue");  
  
    // Scores  
    String scoreslabel = "";  
    for (int score in scores) {  
        scoreslabel += score.toString() + " ";  
    }  
    print("Scores\t\t: $scoreslabel");  
}
```

```
void main() {  
    printStudent(name: "Ronan", scores: [15, 20, 45]);  
}
```

Named parameters (optional by default)

Required (not optional)

Default value, if not specified

For-in loop on the iterable list of integer

Address is a nullable type

Nullable state must be checked

4 ways to manage **nullable** data

type? data;

- **NULL ASSERTION** OPERATOR

```
print(data!);
```

I guarantee
that data is
NOT null

Treat it as a
non null type,

- **CLASSIC** CONDITINAL BRANCH

```
if (data != null) {  
    print(data);  
} else {  
    print("default value");  
}
```

- **TERNARY** OPERATOR

```
print(data != null ? data : "default value");
```

- **NULL-COALESCING** OPERATOR

```
print(data?? "default value");
```



Session Objectives



JUNIOR LEVEL

- ✓ Create a **class** with **attributes** and **constructors**
- ✓ Add **methods**
- ✓ Build **aggregation of objects**
- ✓ Create **computed attributes** (get)



NINJA LEVEL

- ✓ Build **immutable** objects
- ✓ Create objects with **optional attributes**
- ✓ Create objects using **different named constructors**
- ✓ Overload the **operator +**

The image shows a metal cookie cutter with the word "Class" written on its flat surface. The cutter is positioned over a light-colored surface, likely a tablecloth, where several round, golden-brown cookies are scattered. Each cookie has the word "Objects" written on it in a dark, sans-serif font. The scene is a metaphor for the relationship between a class and its objects in programming.

A CLASS

IS A

BLUEPRINT

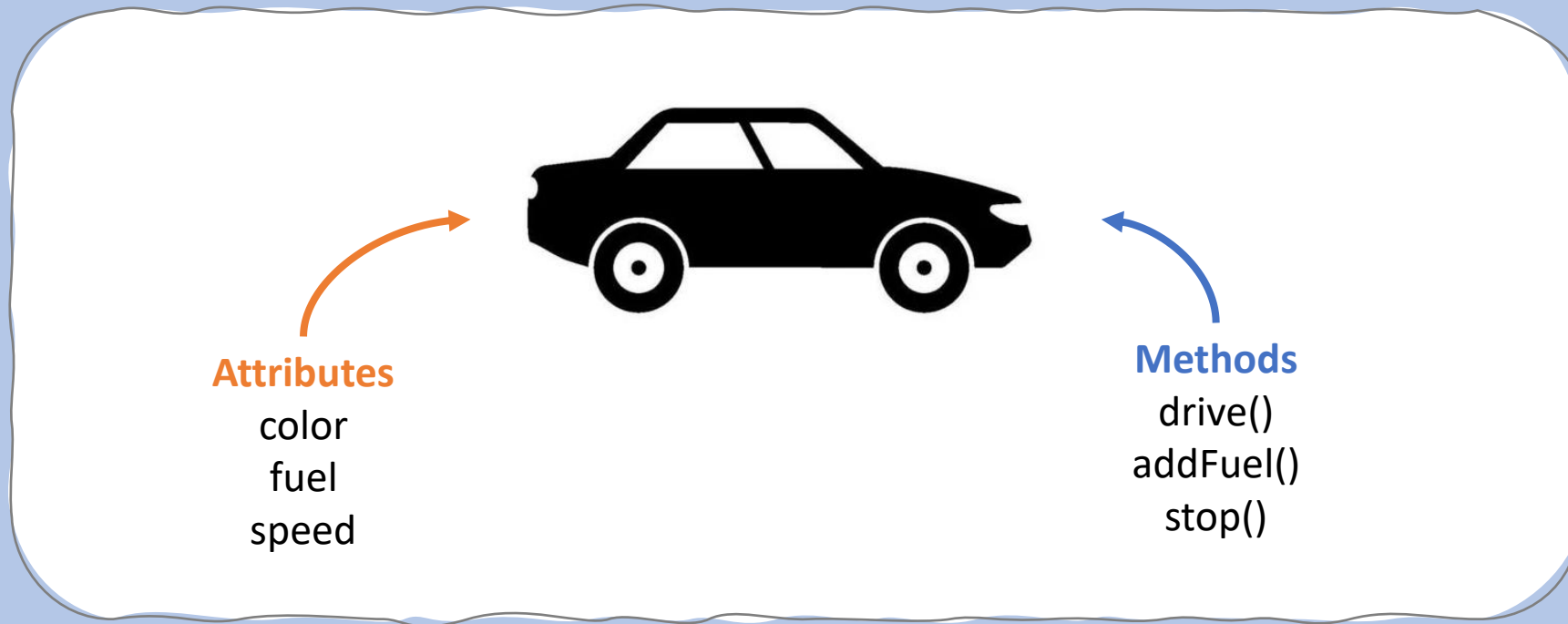
FOR

OBJECTS

Class

Objects

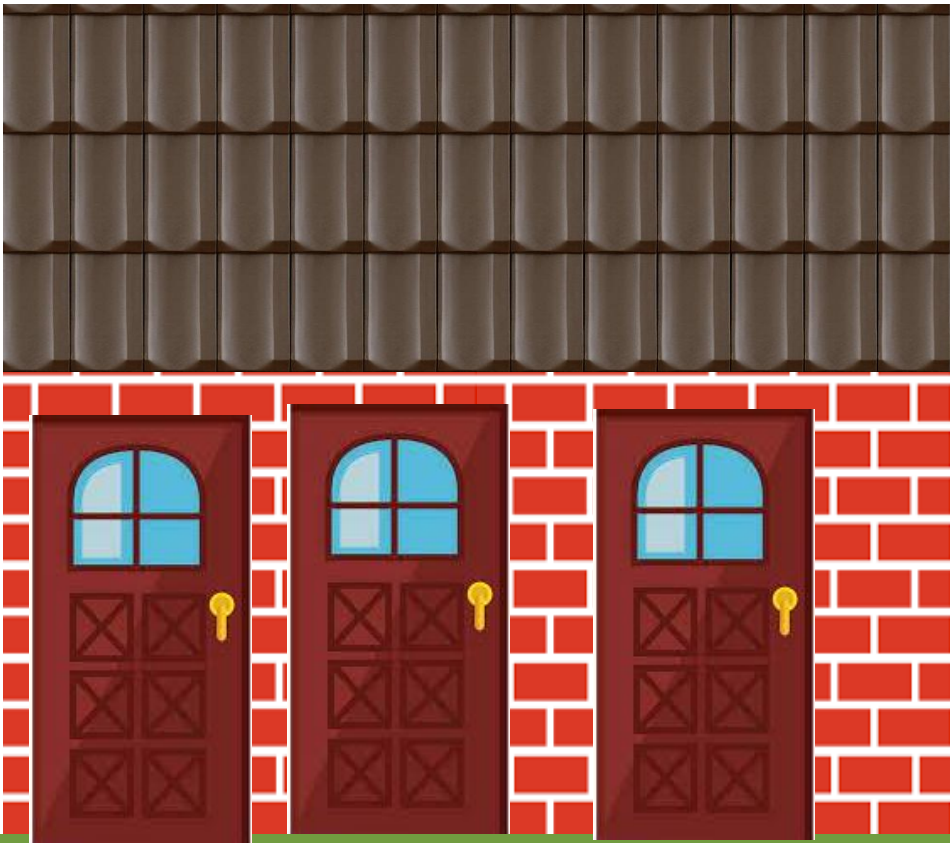
Dart is an **object-oriented** language



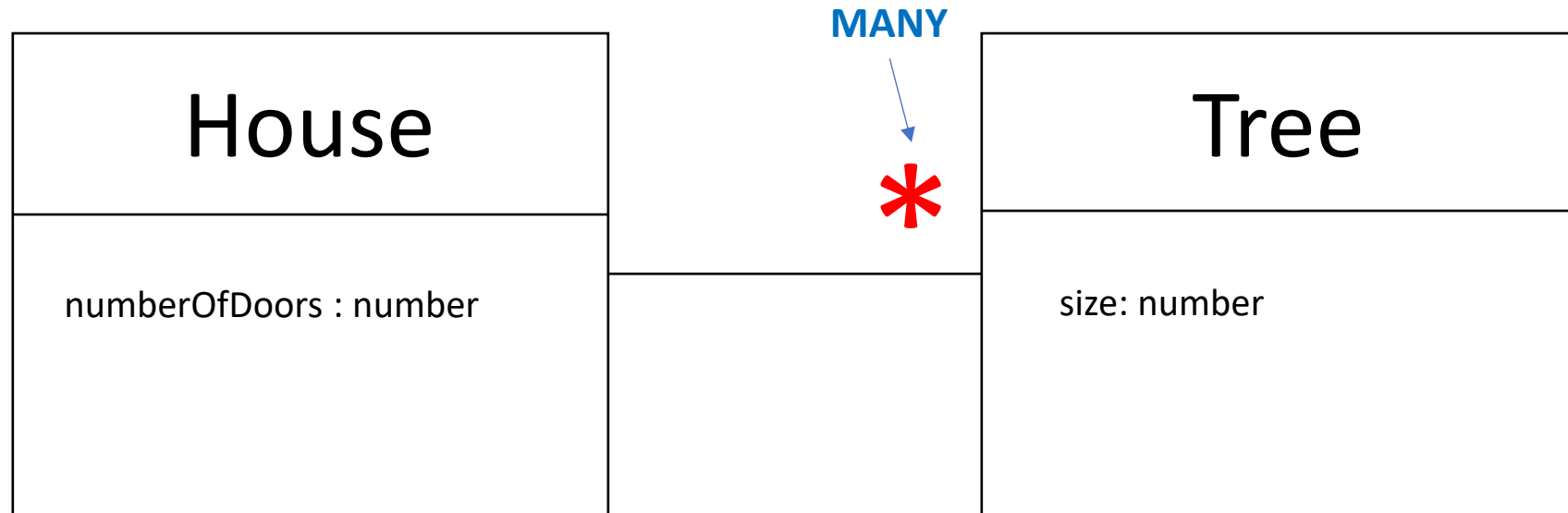
Thanks to classes, I can gather my car attributes and functions in a same place

A house can have **MANY** doors and trees

Each **tree** has a specific **SIZE**



We represent this situation with the bellow **UML Class diagram**:



In **DART**, we represent these 2 classes as follows:

```
class House {  
  int numberOfDoors;  
  List<Tree> trees;  
  
  House(this.numberOfDoors, this.trees);  
}
```

```
class Tree {  
  double height;  
  
  Tree(this.height);  
}
```



- Where are the **constructors**?
- Where are the **class attributes**?

Define and use classes in Dart

DEFINE CLASS

All classes except *Null* descend from **Object**.

class

```
class House {  
  int rooms;
```

Class Attribute
By default public...
Can be modified outside the class!

Constructor

```
House(this.rooms);
```

Class Method

```
bool isBig() {  
  return this.rooms > 5;  
}
```

USE CLASS - INSTANTIATION

```
main() {  
  House frenchHouse = House(2);  
  
  House khmerHouse = House(10);  
  khmerHouse.rooms = 5;  
  
  print(khmerHouse.isBig());  
}
```

Create a house With 2 rooms

Create a house With 10 rooms

We can change x attribute here ... as it's public

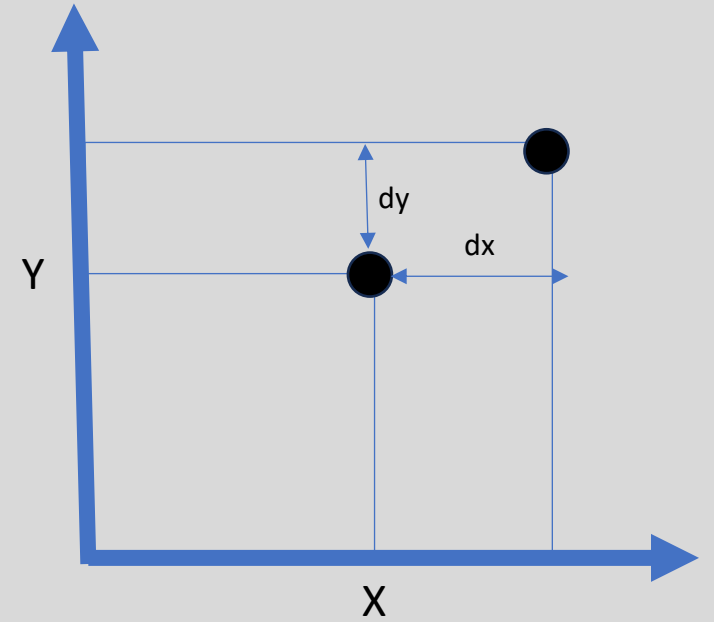
We invoke a method on an object



• What will this code output?

Create a class **Point**

- ✓ A point has a **X and Y** position
- ✓ The class should provide a **method to translate** the point (dx, dy)



OBJECTIVES

- ✓ Create a **class** with attributes
- ✓ Define the **constructor**
- ✓ Add **methods**

SOME HELP !

<https://dart.dev/language/classes>

CORRECTION

```
class Point {  
    int x;  
    int y;  
  
    Point(this.x, this.y);  
  
    @override  
    String toString() {  
        return "x= $x - y= $y";  
    }  
    void translate(int dx, int dy) {  
        x+=dx;  
        y+-dy;  
    }  
}
```

Run | Debug

```
main() {  
    Point p1 = Point(1, 2);  
    print(p1);  
    p1.translate(10, 10);  
    print(p1);  
}
```

- ✓ Create a **class** with attributes
- ✓ Define the **constructor**
- ✓ Add **method**

3 way to manage class **constructors**

```
Point(int x, int y) {  
    this.x = x;  
    this.y = y;  
}
```

Constructor body

Initializer list

```
Point(int x, int y): this.x=x, this.y=y;
```

```
Point(this.x, this.y);
```

Short form constructor !